Python FastAPI Syllabus

1. FastAPI Fundamentals

Topics:

- **Introduction to FastAPI**
    - What is FastAPI?
    - Why FastAPI over Flask/Django?
    - FastAPI architecture
    - ASGI vs WSGI
- **Installation and Setup**
    - Installing FastAPI
    - Installing Uvicorn server
    - Creating first FastAPI project

Example:

pip install fastapi uvicorn

Project structure:

app/
├── main.py
├── routers/
├── models/
├── schemas/
├── database/

- └── services/

Practice:

Build:

- Hello World API
- Basic calculator API
- Simple user API

2. API Fundamentals

Topics:

Understand HTTP deeply:

- HTTP methods
    - GET
    - POST
    - PUT
    - PATCH
    - DELETE
- HTTP Status Codes
    - 200 Success
    - 201 Created
    - 400 Bad Request
    - 401 Unauthorized
    - 404 Not Found
    - 500 Server Error
- REST API principles
- JSON request and response

Practice:

Create:

- Product API
- Student API
- Employee API

3. Routing in FastAPI

Topics:

- Creating API routes

Example:

```
@app.get("/users")
def get_users():
```

return users

Learn:

- Path parameters

Example:

/users/{id}

- Query parameters

Example:

/users?page=1&limit=10

- Multiple routers

Example:

users router
products router
orders router

Practice:

Create:

- User management API
- Product management API

4. Pydantic Models and Data Validation

(One of the most important FastAPI concepts)

Topics:

- Introduction to Pydantic
- Request models
- Response models
- Data validation
- Optional fields
- Default values

- Custom validators

Example:

```
class User(BaseModel):
    name:str
    age:int
    email:str
```

Practice:

Create:

- Registration API
- Profile update API

5. CRUD Operations

(Real project foundation)

Topics:

Learn complete CRUD:

Create

POST /users

Read

GET /users

Update

PUT /users/{id}

Delete

DELETE /users/{id}

Practice:

Build:

- Task Management API
- Blog API
- Inventory API

6. Database Integration

SQL Database

Topics:

Learn:

- SQL basics
- Database connection
- ORM concept

Databases:

- PostgreSQL (Recommended)
- MySQL

Tools:

- SQLAlchemy
- SQLModel

Learn:

- Models
- Tables
- Relationships
- Queries

Example:

User
|
|
Orders

Database relationships:

- One-to-One
- One-to-Many
- Many-to-Many

Practice:

Build:

- E-commerce backend
- Employee management system

7. Database Migration

Topics:

Learn:

- Alembic

Commands:

alembic init
alembic revision
alembic upgrade

Understand:

- Migration files
- Database version control

Practice:

Create database migrations for:

- Users
- Products
- Orders

8. Authentication and Security

(Required for professional projects)

Topics:

Authentication:

- User registration
- Login system
- Password hashing

Libraries:

- Passlib
- Bcrypt

Learn:

- JWT Authentication

Flow:

Login
↓
Generate Token
↓
Send Token
↓
Verify User

Security:

- OAuth2
- Access token
- Refresh token

Practice:

Build:

Complete authentication system:

- Register
- Login
- Logout

- Protected APIs

9. Dependency Injection

(FastAPI special feature)

Topics:

Learn:

- Depends()

Use cases:

- Database connection
- Authentication
- Permissions
- Common functions

Example:

Depends(get_current_user)

Practice:

Create:

- Admin permission system
- User role management

10. Middleware and Request Handling

Topics:

Learn:

- Middleware concept

Implement:

- CORS
- Logging middleware
- Request tracking

- Error handling

Example:

Frontend:

React / Next.js
|
|
FastAPI

Practice:

Create:

- API request logger
- Global error handler

11. File Upload and Media Handling

Topics:

Learn:

- Upload files
- Images
- Documents

Features:

- Upload validation
- File size checking
- Storage handling

Storage:

- Local storage
- AWS S3 basics

Practice:

Build:

- Profile image upload
- Document management system

12. Background Tasks

Topics:

Learn:

- BackgroundTasks

Use cases:

- Sending emails
- Notifications
- Report generation

Advanced:

- Celery
- Redis Queue

Practice:

Build:

- Email notification system
- Report generator

13. Async Programming in FastAPI

Topics:

Python async:

- async / await
- asyncio basics

FastAPI:

- Async endpoints
- Async database operations

Example:

```
async def get_users():
    pass
```

Practice:

Create:

- Async API calls
- High-performance API

14. WebSockets

Topics:

Learn:

- Real-time communication

Use cases:

- Chat applications
- Live notifications
- Tracking systems

Practice:

Build:

- Real-time chat API
- Live notification system

15. API Documentation

Topics:

FastAPI automatic documentation:

- Swagger UI
- ReDoc

Learn:

- API descriptions
- Response examples
- Schema documentation

Practice:

Document complete API project.

16. Testing FastAPI Applications

Topics:

Learn:

- pytest

Testing:

- API endpoints
- Database testing
- Authentication testing

Practice:

Write tests for:

- User APIs
- Product APIs
- Authentication APIs

17. Advanced FastAPI Concepts

Topics:

- Custom exception handling
- Logging
- Rate limiting
- API versioning
- Pagination
- Filtering
- Sorting

- Search functionality

Practice:

Build:

Production-level REST API

18. FastAPI with Frontend Integration

Topics:

Connect:

Next.js / React
|
|
FastAPI
|
|
PostgreSQL

Learn:

- CORS configuration
- API calls
- Authentication flow
- Token handling
- Environment variables

Practice:

Build:

Full-stack application:

- CRM system
- Project management tool
- ERP system

19. Deployment and DevOps

Topics:

Server:

- Linux basics
- VPS setup

Production:

- Uvicorn
- Gunicorn
- Nginx

Containers:

- Docker
- Docker Compose

CI/CD:

- GitHub Actions

Monitoring:

- Logs
- Error tracking

Practice:

Deploy FastAPI project on:

- VPS
- AWS
- Docker

20. Final Production Project

Build one complete backend system:

ERP / Project Management Backend

Features:

- User authentication
- Role permissions
- Employee management
- Task management
- Project tracking
- File uploads
- Notifications
- Reports
- PostgreSQL database
- API documentation
- Docker deployment

Recommended Learning Order

Python Fundamentals

↓

OOP

↓

SQL Database

↓

REST API Concepts

↓

FastAPI Basics

↓

Pydantic

↓

CRUD

↓

PostgreSQL + SQLAlchemy

↓

Authentication

↓

Advanced FastAPI

↓

Docker + Deployment

↓

Production Project